

FLUJEX: Finanzas Personales y flujo de caja

Aplicación Web Responsiva con Base de Datos Normalizada

Proyecto del Curso de Bases de Datos

25 de marzo de 2026

Índice

1. Introducción	3
1.1. Objetivos del Proyecto	3
1.2. Alcance	3
2. Análisis de Requisitos	3
2.1. Requisitos Funcionales	3
2.2. Requisitos No Funcionales	4
3. Diseño de la Base de Datos	4
3.1. Entidades Identificadas	4
3.2. Diagrama Entidad-Relación (ER)	5
4. Normalización de la Base de Datos	6
4.1. Primera Forma Normal (1FN)	6
4.2. Segunda Forma Normal (2FN)	6
4.3. Tercera Forma Normal (3FN)	7
4.4. Diagrama de Dependencias Funcionales	8
5. Scripts SQL	8
5.1. Creación de Tablas	8
5.2. Índices para Optimización	10
6. Consultas Típicas	11
6.1. Obtener Gastos del Usuario Actual	11
6.2. Resumen de Gastos por Categoría	11
6.3. Balance de Cuentas	11

7. Arquitectura de la Aplicación	12
7.1. Tecnologías Recomendadas	12
7.2. Arquitectura de Capas	12
8. Responsividad	13
8.1. Puntos de Quiebre (Breakpoints)	13
8.2. Principios de Diseño Responsivo	13
9. Seguridad	13
9.1. Medidas de Seguridad Implementadas	13
9.2. Ejemplo: Hash de Contraseña	14
10. Plan de Implementación	14
10.1. Fases del Proyecto	14
10.1.1. Fase I	14
10.1.2. Fase II	14
10.1.3. Fase III	14
10.1.4. Fase IV	14
11. Aspectos Administrativos	15
12. Conclusión	15
A. Bibliografía	15

1. Introducción

El presente proyecto tiene como objetivo desarrollar una aplicación web responsiva para la gestión de finanzas personales. La aplicación permitirá a los usuarios registrar, clasificar y analizar sus gastos de manera intuitiva y eficiente, accesible tanto desde dispositivos móviles como desde computadoras de escritorio.

1.1. Objetivos del Proyecto

- Diseñar una base de datos normalizada para almacenar información financiera personal
- Implementar una aplicación web responsiva usando tecnologías modernas
- Permitir el registro, actualización y eliminación de gastos
- Generar reportes y análisis de gastos por categoría y período
- Garantizar la seguridad y privacidad de los datos del usuario

1.2. Alcance

El proyecto se enfoca en:

1. Diseño de la base de datos (BD)
2. Normalización de la BD (hasta 3FN)
3. Arquitectura de la aplicación web
4. Responsividad en dispositivos móviles y de escritorio
5. Documentación técnica completa

2. Análisis de Requisitos

2.1. Requisitos Funcionales

1. **Gestión de Usuarios:** Registro, autenticación y perfil de usuario
2. **Registro de Gastos:** Crear, leer, actualizar y eliminar (CRUD) gastos
3. **Categorización:** Clasificar gastos por categorías personalizables
4. **Reportes:** Generar reportes de gastos por período y categoría
5. **Dashboard:** Visualizar resumen de finanzas en tiempo real
6. **Búsqueda y Filtrado:** Localizar gastos específicos fácilmente

2.2. Requisitos No Funcionales

- **Responsividad:** Compatible con pantallas de 320px a 2560px
- **Seguridad:** Encriptación de contraseñas y validación de datos ¿Hashing?
- **Rendimiento:** Carga de páginas menor a 2 segundos
- **Disponibilidad:** Uptime mínimo del 99.5 % (Sujeto a concurrencia, no se va a probar con cientos de usuarios)
- **Usabilidad:** Interfaz intuitiva y accesible

3. Diseño de la Base de Datos

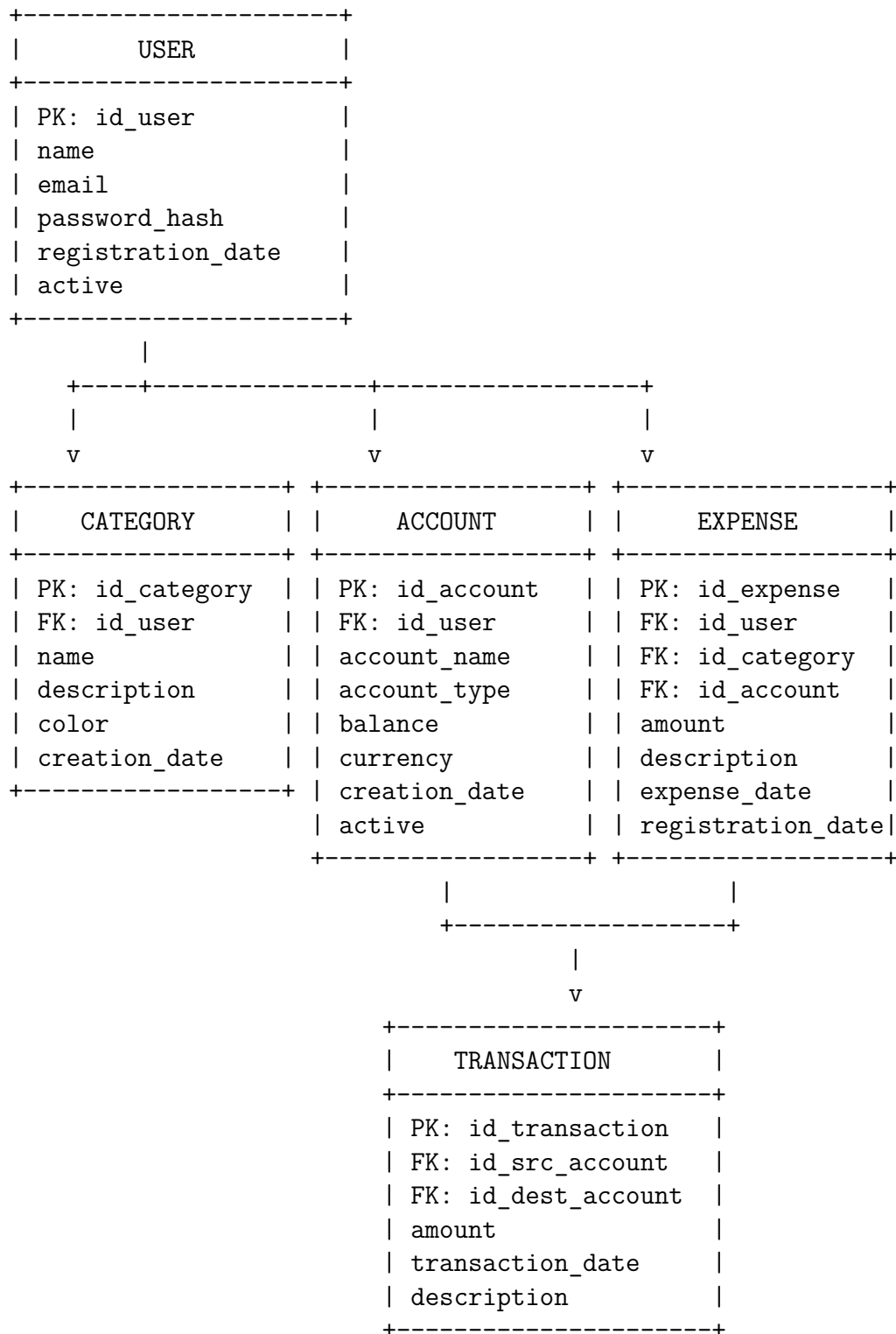
3.1. Entidades Identificadas

Se han identificado las siguientes entidades principales:

1. **Usuario:** Información del usuario (nombre, email, contraseña)
2. **Categoría:** Tipos de gastos (Alimentación, Transporte, etc.), esto sería el equivalente a cuenta en un análisis financiero
3. **Gasto:** Registros individuales de gastos
4. **Cuenta:** Cuentas bancarias o billeteras del usuario
5. **Transacción:** Movimientos financieros entre cuentas

3.2. Diagrama Entidad-Relación (ER)

A continuación se presenta el diagrama ER que muestra las relaciones entre las entidades:



4. Normalización de la Base de Datos

4.1. Primera Forma Normal (1FN)

La primera forma normal requiere que todos los atributos contengan valores atómicos (indivisibles).

Verificación:

- ✓ Cada atributo contiene un único valor
- ✓ No existen grupos repetidos
- ✓ Cada fila es única

Ejemplo – Tabla USUARIO:

```
CREATE TABLE USUARIO (  
    id_usuario    INT          PRIMARY KEY AUTO_INCREMENT,  
    nombre        VARCHAR(100) NOT NULL,  
    email         VARCHAR(100) UNIQUE NOT NULL,  
    passwd_hash   VARCHAR(255) NOT NULL,  
    fecha_reg     TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,  
    activo        BOOLEAN      DEFAULT TRUE  
);
```

4.2. Segunda Forma Normal (2FN)

La segunda forma normal requiere que la tabla esté en 1FN y que todos los atributos no llave dependan completamente de la clave primaria.

Verificación:

- ✓ Tabla está en 1FN
- ✓ No existen dependencias parciales
- ✓ Todos los atributos dependen de la clave primaria completa

Tabla GASTO (ejemplo):

En la tabla GASTO, cada atributo depende completamente de `id_gasto`:

- El monto depende del gasto específico
- La categoría depende del gasto específico
- La fecha depende del gasto específico

```
CREATE TABLE GASTO (  
    id_gasto      INT          PRIMARY KEY AUTO_INCREMENT,  
    id_usuario    INT          NOT NULL,  
    id_categoria  INT          NOT NULL,  
    id_cuenta     INT          NOT NULL,  
    monto         DECIMAL(10, 2) NOT NULL,  
    descripcion   VARCHAR(255),  
    fecha_gasto   DATE          NOT NULL,  
    fecha_reg     TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (id_usuario) REFERENCES USUARIO(id_usuario),  
    FOREIGN KEY (id_categoria) REFERENCES CATEGORIA(id_categoria),  
    FOREIGN KEY (id_cuenta) REFERENCES CUENTA(id_cuenta)  
);
```

4.3. Tercera Forma Normal (3FN)

La tercera forma normal requiere que la tabla esté en 2FN y que no existan dependencias transitivas de atributos llave a atributos no llave.

Verificación:

- ✓ Tabla está en 2FN
- ✓ No existen dependencias transitivas
- ✓ Los atributos no clave no dependen de otros atributos no clave

Ejemplo de violación evitada:

INCORRECTO – Guardar el nombre del usuario en GASTO:

```
-- Viola 3FN: nombre_usuario depende transitivamente  
CREATE TABLE GASTO_MAL (  
    id_gasto      INT PRIMARY KEY,  
    nombre_usuario VARCHAR(100), -- Dependencia transitiva!  
    ...  
);
```

CORRECTO – Mantener la referencia a la tabla USUARIO:

```
-- Cumple 3FN  
CREATE TABLE GASTO (  
    id_gasto      INT PRIMARY KEY,  
    id_usuario    INT NOT NULL,    -- Solo la FK, no el nombre  
    ...  
    FOREIGN KEY (id_usuario) REFERENCES USUARIO(id_usuario)  
);
```

4.4. Diagrama de Dependencias Funcionales

$\text{id_gasto} \rightarrow \text{id_usuario}, \text{id_categoria}, \text{id_cuenta}, \text{monto}, \text{descripcion}, \text{fecha_gasto}$

Donde:

- $\text{id_usuario} \rightarrow \text{nombre}, \text{email}, \text{passwd_hash}$ (tabla USUARIO)
- $\text{id_categoria} \rightarrow \text{nombre}, \text{descripcion}, \text{color}$ (tabla CATEGORIA)
- $\text{id_cuenta} \rightarrow \text{nombre_cuenta}, \text{saldo}, \text{moneda}$ (tabla CUENTA)

5. Scripts SQL

5.1. Creación de Tablas

```
-----  
-- Tabla USUARIO  
-----  
CREATE TABLE USUARIO (  
    id_usuario      INT          PRIMARY KEY AUTO_INCREMENT,  
    nombre          VARCHAR(100) NOT NULL,  
    email           VARCHAR(100) UNIQUE NOT NULL,  
    passwd_hash     VARCHAR(255) NOT NULL,  
    telefono        VARCHAR(20),  
    fecha_reg       TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,  
    ultimo_acceso   TIMESTAMP,  
    activo          BOOLEAN      DEFAULT TRUE,  
    INDEX idx_email (email)  
);  
  
-----  
-- Tabla CATEGORIA  
-----  
CREATE TABLE CATEGORIA (  
    id_categoria    INT          PRIMARY KEY AUTO_INCREMENT,  
    id_usuario      INT          NOT NULL,  
    nombre          VARCHAR(50)  NOT NULL,  
    descripcion     VARCHAR(255),  
    color           VARCHAR(7),  
    icono           VARCHAR(50),  
    fecha_reg       TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (id_usuario) REFERENCES USUARIO(id_usuario)  
        ON DELETE CASCADE,  
    UNIQUE KEY uq_categoria (id_usuario, nombre),  
    INDEX idx_usuario (id_usuario)
```

```
);

-----
-- Tabla CUENTA
-----

CREATE TABLE CUENTA (
    id_cuenta      INT          PRIMARY KEY AUTO_INCREMENT,
    id_usuario     INT          NOT NULL,
    nombre         VARCHAR(100) NOT NULL,
    tipo           ENUM('Efectivo','Banco',
                       'Tarjeta','Billetera') NOT NULL,
    saldo          DECIMAL(12,2) DEFAULT 0.00,
    moneda         VARCHAR(3)   DEFAULT 'USD',
    descripcion    VARCHAR(255),
    fecha_reg      TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    activa         BOOLEAN      DEFAULT TRUE,
    FOREIGN KEY (id_usuario) REFERENCES USUARIO(id_usuario)
        ON DELETE CASCADE,
    INDEX idx_usuario (id_usuario)
);

-----
-- Tabla GASTO
-----

CREATE TABLE GASTO (
    id_gasto       INT          PRIMARY KEY AUTO_INCREMENT,
    id_usuario     INT          NOT NULL,
    id_categoria   INT          NOT NULL,
    id_cuenta      INT          NOT NULL,
    monto          DECIMAL(10,2) NOT NULL,
    descripcion    VARCHAR(255),
    fecha_gasto    DATE         NOT NULL,
    etiquetas      VARCHAR(255),
    comprobante    VARCHAR(500),
    fecha_reg      TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    fecha_upd      TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (id_usuario)
        REFERENCES USUARIO(id_usuario) ON DELETE CASCADE,
    FOREIGN KEY (id_categoria)
        REFERENCES CATEGORIA(id_categoria) ON DELETE RESTRICT,
    FOREIGN KEY (id_cuenta)
        REFERENCES CUENTA(id_cuenta) ON DELETE RESTRICT,
    INDEX idx_usr_fecha (id_usuario, fecha_gasto),
    INDEX idx_categoria (id_categoria),
    INDEX idx_cuenta    (id_cuenta)
);
```

```
-- Tabla TRANSACCION

CREATE TABLE TRANSACCION (
    id_transaccion INT PRIMARY KEY AUTO_INCREMENT,
    id_usuario INT NOT NULL,
    id_cta_origen INT NOT NULL,
    id_cta_destino INT,
    tipo ENUM('Gasto', 'Ingreso', 'Transferencia') NOT NULL,
    monto DECIMAL(10,2) NOT NULL,
    descripcion VARCHAR(255),
    fecha_trans DATETIME NOT NULL,
    fecha_reg TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_usuario)
        REFERENCES USUARIO(id_usuario) ON DELETE CASCADE,
    FOREIGN KEY (id_cta_origen)
        REFERENCES CUENTA(id_cuenta) ON DELETE RESTRICT,
    FOREIGN KEY (id_cta_destino)
        REFERENCES CUENTA(id_cuenta) ON DELETE SET NULL,
    INDEX idx_usr_fecha (id_usuario, fecha_trans)
);
```

5.2. Índices para Optimización

```
-- Indices adicionales para mejorar rendimiento

CREATE INDEX idx_gasto_mes
    ON GASTO(id_usuario, MONTH(fecha_gasto));

CREATE INDEX idx_gasto_ano
    ON GASTO(id_usuario, YEAR(fecha_gasto));

CREATE INDEX idx_categoria_usuario ON CATEGORIA(id_usuario);
CREATE INDEX idx_cuenta_usuario ON CUENTA(id_usuario);
```

6. Consultas Típicas

6.1. Obtener Gastos del Usuario Actual

```
SELECT
    g.id_gasto,
    g.descripcion,
    g.monto,
    c.nombre AS categoria,
    cu.nombre AS cuenta,
    g.fecha_gasto
FROM GASTO g
JOIN CATEGORIA c ON g.id_categoria = c.id_categoria
JOIN CUENTA cu ON g.id_cuenta = cu.id_cuenta
WHERE g.id_usuario = ?
ORDER BY g.fecha_gasto DESC;
```

6.2. Resumen de Gastos por Categoría

```
SELECT
    c.nombre AS categoria,
    COUNT(g.id_gasto) AS cantidad,
    SUM(g.monto) AS total,
    AVG(g.monto) AS promedio
FROM GASTO g
JOIN CATEGORIA c ON g.id_categoria = c.id_categoria
WHERE g.id_usuario = ?
    AND MONTH(g.fecha_gasto) = MONTH(CURDATE())
    AND YEAR(g.fecha_gasto) = YEAR(CURDATE())
GROUP BY c.id_categoria, c.nombre
ORDER BY total DESC;
```

6.3. Balance de Cuentas

```
SELECT
    c.nombre AS cuenta,
    c.tipo,
    c.saldo,
    c.moneda,
    COUNT(DISTINCT g.id_gasto) AS num_gastos
FROM CUENTA c
LEFT JOIN GASTO g ON c.id_cuenta = g.id_cuenta
WHERE c.id_usuario = ?
GROUP BY c.id_cuenta
ORDER BY c.fecha_reg DESC;
```

7. Arquitectura de la Aplicación

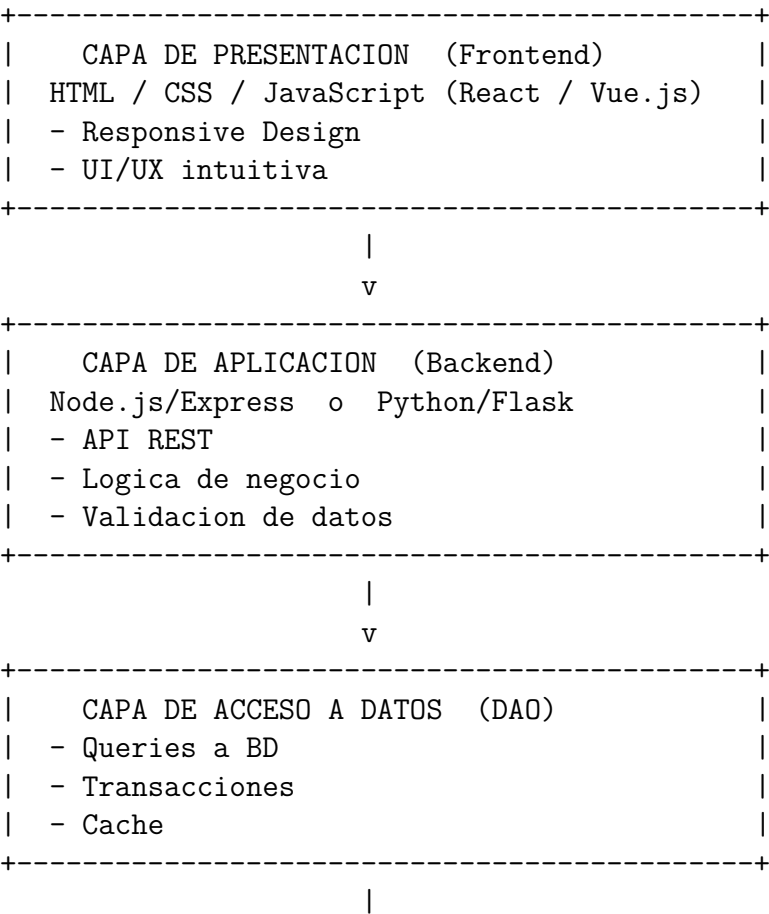
7.1. Tecnologías Recomendadas

Esta lista de tecnologías, son sugerencias, pero no deben considerarse obligatorias, salvo el motor de BD que sea MySQL, sin embargo, son recomendadas en particular porque se cuenta con mucha documentación y apoyo de IA (para poder hacer gran parte del desarrollo con esta herramienta), la cual no debe sustituir el desarrollo de habilidades sobre las consultas de la BD.

Capa	Tecnología	Justificación
Backend	Node.js/Express o Python/Flask	Escalable y fácil de mantener
Frontend	React / Vue.js	Responsivo y componentes reutilizables
Base de Datos	MySQL 8.0+	Robusta, relacional, amplio soporte
Autenticación	JWT	Seguro y sin estado (stateless)
CSS Framework	Bootstrap / Tailwind	Responsividad garantizada

Cuadro 1: Stack tecnológico propuesto

7.2. Arquitectura de Capas



v

CAPA DE DATOS (Base de Datos)
MySQL 8.0+
- Almacenamiento persistente
- Integridad referencial

8. Responsividad

8.1. Puntos de Quiebre (Breakpoints)

Dispositivo	Ancho	Características
Móvil	320px – 480px	Una columna, botones grandes
Tablet	481px – 768px	Dos columnas, navegación adaptada
Laptop	769px – 1024px	Tres columnas, sidebar
Desktop	1025px+	Diseño completo

Cuadro 2: Breakpoints de diseño responsivo

8.2. Principios de Diseño Responsivo

- 1. **Mobile First:** Diseñar primero para móvil
- 2. **Flexible Grids:** Usar CSS Grid y Flexbox
- 3. **Flexible Images:** Imágenes adaptativas
- 4. **Media Queries:** CSS adaptable a diferentes tamaños

9. Seguridad

9.1. Medidas de Seguridad Implementadas

- **Encriptación de Contraseñas:** Usar bcrypt o Argon2
- **HTTPS:** Todos los datos encriptados en tránsito
- **Validación de Entrada:** Prevenir inyección SQL
- **CORS:** Control de acceso entre dominios
- **Rate Limiting:** Limitar intentos de login
- **Tokens JWT:** Autenticación sin estado

9.2. Ejemplo: Hash de Contraseña

```
-- Las contraseñas NUNCA se guardan en texto plano
UPDATE USUARIO
SET     passwd_hash = bcrypt('mi_contraseña')
WHERE   id_usuario  = 1;
```

10. Plan de Implementación

10.1. Fases del Proyecto

Fase	Duración	Actividades
1	Semana 1	Diseño BD, normalización, scripts SQL
2	Semana 2	Backend: APIs CRUD, autenticación
3	Semana 3	Frontend: Interfaz responsiva
4	Semana 4	Testing, documentación, presentación

Cuadro 3: Cronograma de implementación

10.1.1. Fase I

Las tablas propuestas en este proyecto, son una simplificación, considerar qué otros atributos deben de poder incorporarse o por ejemplo, si una persona tiene entradas de dinero, cómo se deberían contabilizar en las cuentas y por ende, cuáles son los activos asociados.

Se busca que el model planteado tenga más entidades y responda a las particularidades del problema y sea extensible (por ejemplo, si se quiere que un agente de IA con un correo reciba las notificaciones de cada transacción de una tarjeta de crédito y automáticamente alimente la BD.

También la prueba de que está normalizado el diseño de la BD.

10.1.2. Fase II

En esta fase, se espera que los estudiantes puedan estudiar con qué tecnologías pueden proteger y trabajar, así como el diseño de clases, siempre pensando en lo aprendido en POO y buscando que sea escalable e integrable con otros módulos.

10.1.3. Fase III

Esta fase se espera al menos un prototipo de como se verían las ventanas de trabajo de cada etapa, la integración de la BD con el back end y las capas de seguridad asociadas.

10.1.4. Fase IV

Esta última etapa es donde van a conjuntar todas las partes de las etapas previas y van a presentar la aplicación así como una prueba de concepto sobre cómo administrar cuentas con la BD.

11. Aspectos Administrativos

- El proyecto puede ser desarrollado en parejas o tríos
- Cada semana se dedicará una hora de clase para revisar el avance de los proyectos.
- El proyecto debe funcionar correctamente en Linux (aka el servidor debe estar en Linux) y en cualquier navegador
- La fecha de entrega final propuesta es 29 de abril.

12. Conclusión

Este proyecto proporciona una base para desarrollar una aplicación de gestión de finanzas personales. La normalización a 3FN garantiza:

- ✓ Integridad de datos
- ✓ Eliminación de redundancia
- ✓ Escalabilidad
- ✓ Mantenibilidad

La arquitectura responsiva asegura que la aplicación sea accesible desde cualquier dispositivo, mejorando la experiencia del usuario y aumentando su utilidad.

A. Bibliografía

Referencias

- [1] Codd, E. F. (1970). *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM.
- [2] Oracle Corporation. *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/>
- [3] Date, C. J. (2003). *An Introduction to Database Systems* (8th ed.). Addison-Wesley.